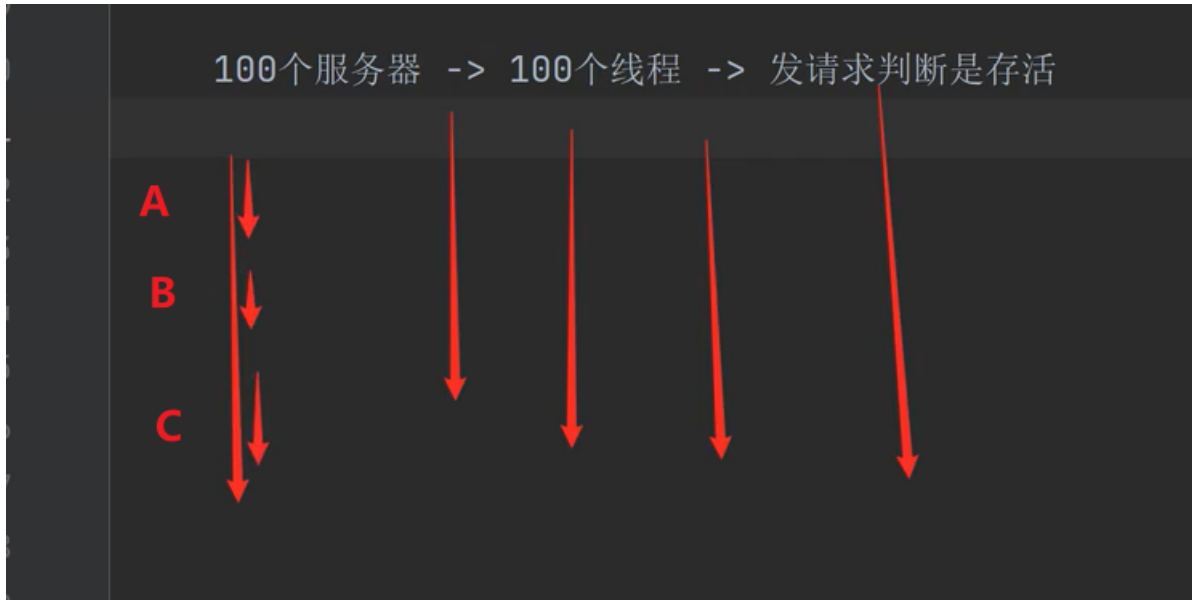


操作系统笔记

学习篇

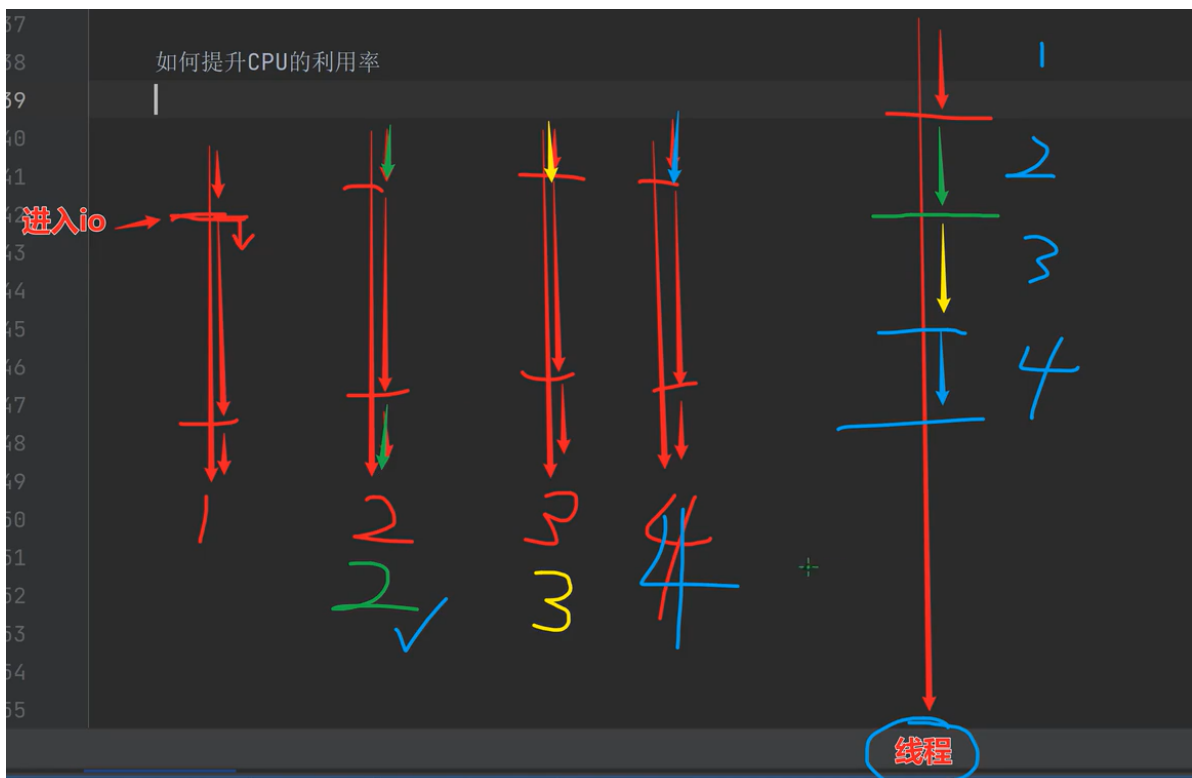
进程与线程

关于线程



A和B类可以是请求、准备参数等等，这些内容是由CPU来处理，速度很快。但是C类属于网络传输等IO内容，速度很慢，这期间CPU就可以转移到下一个线程来进行处理，从而提高效率

关于协程



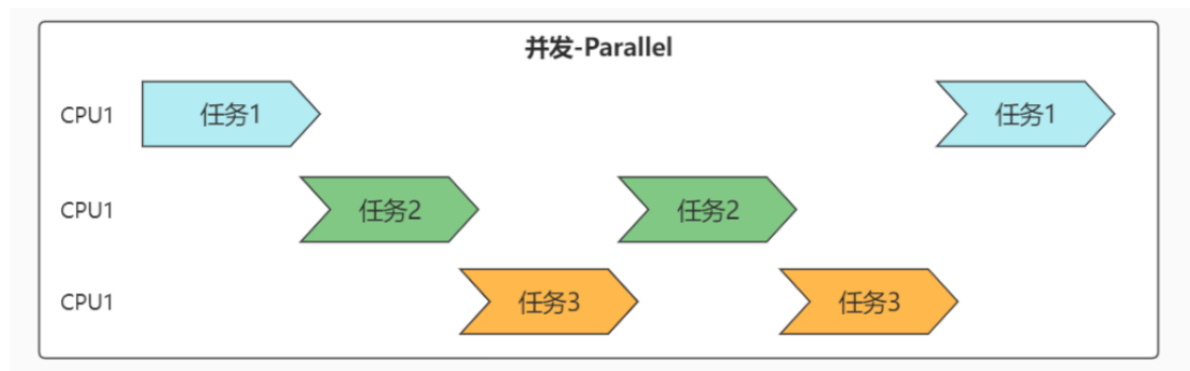
将所有除IO内容以外的内容（也就是由CPU计算的内容）归并到一个线程上去

协程语法内容：

由asyncs申明的协程函数，得到的是一个协程对象，而不是直接运行该函数

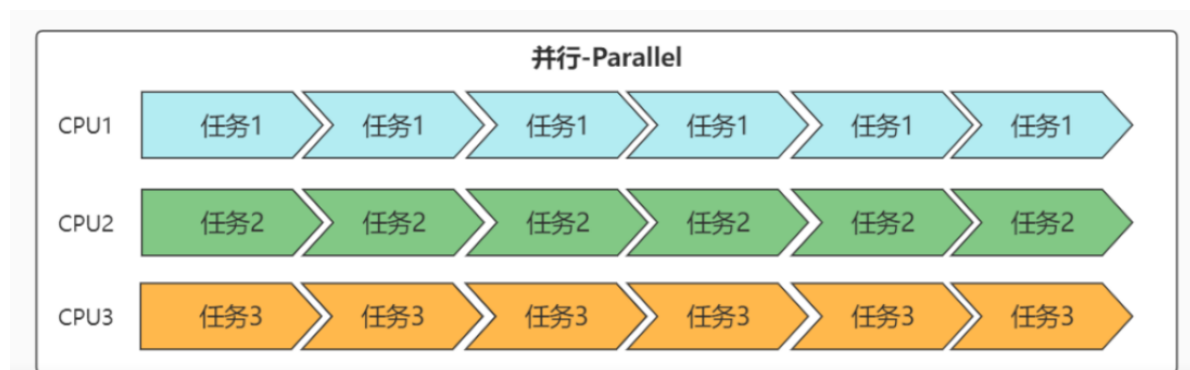
关于并发

单CPU在一个时间段只执行一个任务，但是因为处理速度非常快，导致时间段非常短，看起来就像是同时执行



关于并行

多个CPU同时执行多个任务



进程管理

如果有大量处于阻塞状态的进程，进程可能会占用着物理内存空间

如果有大量处于阻塞状态的进程，进程可能会占用着物理内存空间，在虚拟内存管理的操作系统中，通常会把阻塞状态的进程的物理内存空间换出到硬盘，等需要再次运行的时候，再从硬盘换到物理内存，这一个新的状态，就叫做挂起，挂起状态分为两种

阻塞挂起状态：进程在外存（硬盘）并等待某个事件的出现

就绪挂起状态：进程在外存（硬盘），但只要进入内存，即刻立刻运行

唤醒进程

如果某进程正在等待I/O事件，需由别的进程发消息给它，则只有当该进程所期待的事件出现时，才由发现者进程唤醒它

“别的进程”是负责感知或触发目标进程所等待的 I/O 事件，并在事件满足时执行唤醒的相关进程

CPU的上下文切换

CPU 寄存器是 CPU 内部一个容量小，但是速度极快的内存

程序计数器则是用来存储 CPU 正在执行的指令位置、或者即将执行的下一条指令位置

过程：先保存前一个任务的CPU寄存器和程序计数器的内容到内存，然后加载新任务的之前保存的上下文到寄存器和程序计数器，跳转到新位置，执行新任务

进程的上下文切换

进程的上下文切换包括很多资源--虚拟内容、栈、全局变量等等

线程管理

线程的必要性

维护进程的系统开销很大，进程切换时浪费时间，而且进程之间无法共享数据

同一个进程内多个线程共享代码段、打开的文件等资源

各个线程之间可以并发执行

线程的缺点：

当进程中的一个线程崩溃时，会导致其所属进程的线程崩溃

线程与进程的比较

核心定义与角色：进程是资源分配的基本单位，线程是CPU调度的基本单位

调度与性能开销：进程切换开销和创建开销大，线程相对来说要小很多

通信与同步：进程间通信依赖管道、消息队列、socket等等，线程共享内存，锁避免数据竞争

隔离性与健壮性：进程之间隔离性高，线程之间隔离性差

线程上下文切换

当两个线程不是属于同一个进程，则切换的过程就跟进程上下文切换一样

当两个线程是属于同一个进程，因为虚拟内存是共享的，只需要切换线程的私有数据、寄存器等不共享的数据

关于调度

当进程被创建时，会进入到就绪队列，操作系统就会从就绪队列选择一个进程运行

当进程发生I/O事件而阻塞时，操作系统就会选择另外一个进程运行

当进程退出结束后，操作系统就从就绪队列选择另外一个进程运行

调度原则

CPU利用率：确保匆忙状态

系统吞吐量：短时间作业进程

周转时间：进程运行+阻塞时间+等待时间的总和

等待时间：就绪队列时间不应该太长

响应时间：交互式系统中的响应时间是衡量调度算法好坏的标准，比如鼠标键盘，对响应要求就很高

调度算法

先来先服务：每次从就绪队列选择最先进入队列的进程。但是不利于短作业

最短作业优先

高响应比优先-权衡短作业和长作业

时间片轮转调度算法：每个进程分配一个时间段，仅运行该进程在该时间段运行

最高优先级调度算法，从就绪队列中选择最高优先级的进程运行

内存管理

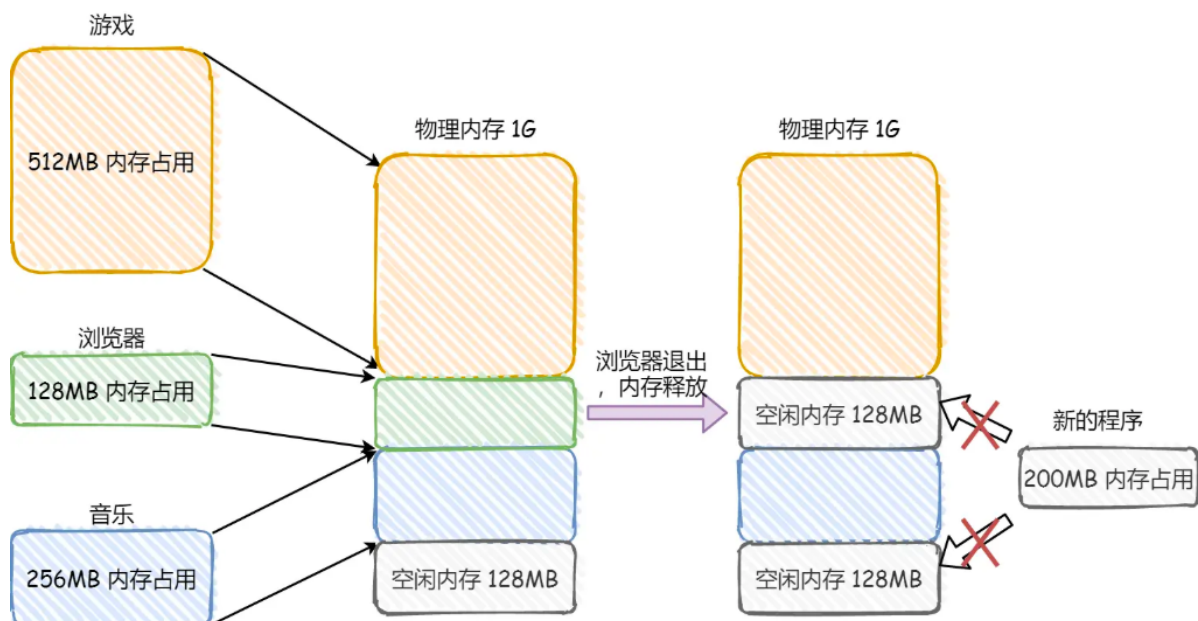
操作系统会将不同进程的虚拟地址和不同内存的物理地址映射起来，虚拟地址都是相同的，由于它们在不同的进程当中，进程中的页写有虚拟地址和物理地址的隐私关系，地址翻译硬件就会完成地址转换

如果程序要访问虚拟地址的时候，由操作系统转换成不同的物理地址，这样不同的进程运行的适合，写入的是不同的物理地址，就不会产生冲突

内存碎片

下面中，一个游戏内存占用部分，内部是没有内存碎片的

但是会产生外部内存碎片，如下图所示



内存交换

第一种：当物理内存很紧张的时候，操作系统会把不常用的内存暂时存放到硬盘，在需要的时候再装回物理内存

第二种：解决的是内存碎片的问题，如果把音乐程序占用的256MB内存写到硬盘上，再从硬盘上读回到内存里，加载到已占用部分的内存后面，就可以空缺出连续的256MB

在Linux系统里，也就是常见的Swap空间，这块空间是从硬盘划分出来的，用于内存与硬盘的空间交换

但是如果内存交换的时候，交换的是一个内存空间很大的程序，机器就可能卡顿，为了解决这个问题，就出现了

内存分页

分配内存的最小单位是一页，所以页内会出现内存浪费，产生内部内存碎片；页与页直接是紧密排列的，所以不会有外部碎片

每一页的完整度很高，也就是可以不同把页一直加载到物理内存，而是在程序运行时，需要用到对应虚拟内存页的指令和数据时，再加载到物理内存里面去

虚拟内存的作用

虚拟内存的实现是依赖于页表的

- 第一，虚拟内存可以使得进程对运行内存超过物理内存大小，因为程序运行符合局部性原理，CPU访问内存会有很明显的重复访问的倾向性，对于那些没有被经常使用到的内存，我们可以把它换出到物理内存之外，比如硬盘上的 swap 区域。
- 第二，由于每个进程都有自己的页表，所以每个进程的虚拟内存空间就是相互独立的。进程也没有办法访问其他进程的页表，所以这些页表是私有的，这就解决了多进程之间地址冲突的问题。
- 第三，页表里的页表项中除了物理地址之外，还有一些标记属性的比特，比如控制一个页的读写权限，标记该页是否存在等。在内存访问方面，操作系统提供了更好的安全性。

内存分配的过程

应用程序申请内存的时候，实际上申请的是虚拟内存，并不会分配物理内存。

当CPU访问这个虚拟内存时，会发现虚拟内存没有映射到物理内存，就会产生缺页中断，进程就会从用户态切换到内核态，缺页中断处理函数会查看是否有空闲的物理内存，如果有，就直接分配物理内存，并建立虚拟内存与物理内存之间的映射关系；如果没有空闲的物理内存，内核就会开始回收内存

回收内存有两种方式：一是后台内存回收，用于内存有压力，但还没到最低阈值的时候，这个过程是异步的，不会阻塞进程的执行；二是直接内存回收，当后台异步回收跟不上进程申请内存的速度时，就会开始直接回收，这个回收过程是同步的，会阻塞进程的执行

如果物理内存任然无法满足申请，就会触发OOM机制，直接杀掉得分高的进程

预读机制

linux在读取文件数据的时候，处于空间局部性原理，会额外读取一些大概率以后会被访问的数据，那么如果后面真的要访问这些数据的时候，就不用再从磁盘去读取，这样就减少了磁盘I/O次数，提高系统磁盘I/O吞吐量

MySQL 中的Buffer pool 也有类似的预读机制，从磁盘加载数据时，会提前把它相邻的页一并加载进来

我觉得，抖音的视频也有一点这个意思，但可能跟多的是减少**网络I/O次数**

但是如果提前加载进来的页一直没有被访问，而且还导致了LRU链表末尾页淘汰（可能是热点数据），这样就降低了缓存命中率，为了解决这个问题，需要让预读页再内存里的时间尽可能短，比如将数据分为冷数据和热数据，分别进行LRU算法（活跃LRU链表和非活跃LRU链表），当非活跃LRU链表中的内容真的被访问到了之后，再加入到活跃LRU链表中去

文件系统

Linux文件系统会为每个文件分配两个数据结构：索引节点和目录项

目录项是内核一个数据结构，缓存在内存；目录是个文件，持久化存储在磁盘

磁盘读写的最小单位是扇区，512B大小，一次性会读写8个扇区，也就是4k

虚拟文件系统

为上层（用户）提供一个统一的接口

磁盘的文件系统：直接把数据存储存储在磁盘中，比如Ext2/3/4,XFS等等

内存的文件系统：直接占用内存空间，比如/proc和/sys文件系统，是读写内核中相关的数据

网络的文件系统：访问其他计算机主机数据的文件系统，比如NFS、SMB等等

文件的存储

连续空间存放，读写效率很高，一次磁盘寻道就可以读出整个文件，但是会产生磁盘空间碎片，文件长度不易扩展

如下图，如果文件B被删除，磁盘上就留下一块空缺，这时，如果新来的文件小于其中的一个空缺，我们就可以将其放在相应空缺里。但如果该文件的大小大于所有的空缺，但却小于空缺大小之和，则虽然磁盘上有足够的空缺，但该文件还是不能存放。当然了，我们可以通过将现有文件进行挪动来腾出空间以容纳新的文件，但是这个在磁盘挪动文件是非常耗时，所以这种方式不太现实。



非连续空间存放，链表的方式：离散的，不连续的，可以消除磁盘碎片，同时文件的长度可以动态扩展，总体都是需要按照链表的顺序去读取物理块，无法做到随机访问

对于隐式链表，采用指针存放下一个数据块的形式，所以存放的方式是可以不连续的，缺点是：指针会消耗一定的存储空间，而且稳定性较差，文件容易丢失，**而且**读取到内存后，才能从这个物理块中获取指向下一个物理块的指针，然后再根据这个指针去读取下一个物理块。也就是说，在查找下一个物理块的过程中，每获取一个新的物理块的指针，都需要先从磁盘把包含该指针的物理块读入内存

对于显式链表，把用于链接文件的指针存放在内存的一张链接表中，查找过程在内存中直接进行，检索速度很快，也减少了访问磁盘的次数

索引的方式：直接读取，无需遍历其他物理块

方式	访问磁盘次数	优点	缺点
顺序分配	需访问磁盘 1 次	顺序存取速度快，当文件是定长时可以根据文件起始地址及记录长度进行随机访问	要求连续的存储空间，会产生外部碎片，不利于文件的动态扩充
链表分配	需访问磁盘 n 次	无外部碎片，提高了外存空间的利用率，动态增长较方便	只能按照文件的指针链顺序访问，查找效率低，指针信息存放消耗内存或磁盘空间
索引分配	m 级需访问磁盘 m+1 次	可以随机访问，易于文件的增删	索引表增加存储空间的开销，索引表的查找策略对文件系统效率影响较大

文件I/O

缓冲I/O：利用的是标准库的缓存实现文件的加速访问

很多程序遇到换行才真正输出，而换行前的内容，实际是被标准库暂时缓存了起来，目的是减少系统调用

磁盘i/o是非常慢的，linux内核会把用户数据拷贝到内核中缓存起来，只有当缓存满足某些条件的时候，才发起I/O磁盘的请求。

设备管理

CPU是通过设备控制器和设备来打交道的

控制器有三类寄存器：状态寄存器、命令寄存器、数据寄存器

面试题篇

用户态和内核态的区别

内核态：CPU可以执行所有的指令和访问的所有的硬件资源，具有更高的权限

用户态：CPU只能执行部分指令集，无法直接访问硬件资源，操作权限低，用于运行用户程序

划分两态原因：

安全性--用户程序无法直接访问硬件资源

稳定性--用户程序出现问题时，不会影响到整个系统

隔离性--有利于模块化和维护

分配给进程的是哪些资源？

内存资源--虚拟地址孔吉纳

CPU资源，细说就是时间片

I/O设备资源--进程运行时需要各种输入输出设备

系统内核资源--进程控制块PCB、各种信号量、消息队列等等

说一下信号量和消息队列：

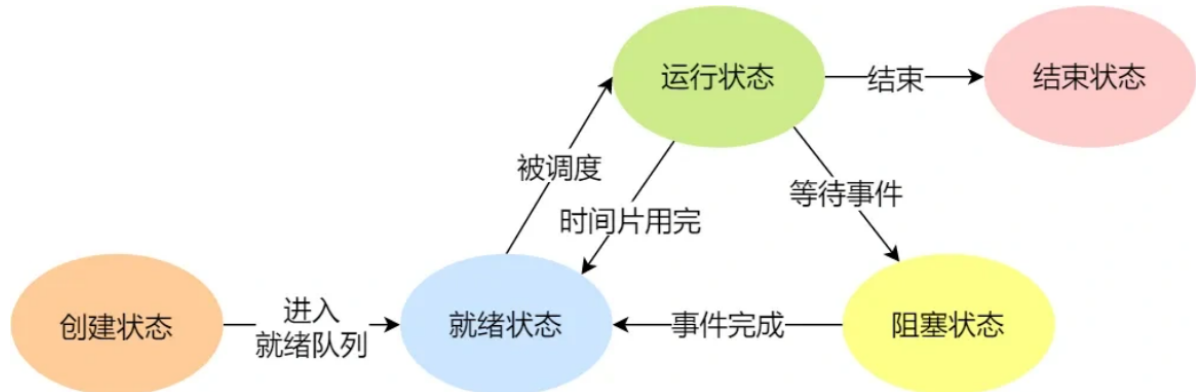
信号量：是用于防止冲突的，P V操作，将信号量的初始值设置为1，就变成了一把锁。通过传递信号，来控制谁进，谁等

消息队列：发送方和接收方不需要同时存在。发送方可以发送完消息就去干别的事，接收方可以在之后任何需要的时候来取消息。主要是实现进程间的信息共享和信息交换

线程切换的步骤

1. 先保存当前线程的上下文信息，包括寄存器、程序计数器、堆栈指针等等
2. 切换到调度器：根据算法决策，选择下一个要执行的线程
3. 下一个要执行的线程选择好后，根据上下文信息恢复该线程的执行状态
4. 调度器将执行权切换到新线程，开始执行

进程的切换状态



死锁发生的条件

1. 互斥条件：多个线程不能同时使用同一个资源
2. 持有并等待，线程A已经有了资源1，想要神奇资源2，但是资源2在线程C那，此时A等待，且不会释放资源1
3. 持有资源不可剥夺
4. 环路等待条件

资源有序分配法，可以破坏环路等待条件

银行家算法：不断检查剩余可用资源是否能够满足某个进程的最大需求。如果有，优先选择这个任务，任务执行完后，回收资源；不断重复这个过程

内存地址转换三步骤

1. 把虚拟内存地址切片分成页号和偏移量
2. 根据页号，从页表里面，查询对应的物理页号
3. 直接拿物理页号，加上前面的偏移量，就得到了物理内存地址

哪些内存是可以回收的？

文件页：大部分文件页，都可以直接释放内存，有需要的时候，再从磁盘重新读取；而那些被应用程序修改过，并且暂时还没有写入磁盘的数据（也就是脏页），得先写入磁盘，然后才能释放内存（txt文档先保存再关闭）

匿名页：这部分内存没有实际载体，跟可能被再次访问，不能直接释放内存，要先写入磁盘再释放

两类页的回收方法都是基于LRU算法（活跃链表和非活跃链表）

网络I/O

指的是计算机通过网络接口（网卡）进行数据读取和写入的过程

数据在网络上的传输速度和CPU的处理速度存在巨大的差距，CPU执行一条指令是纳秒级别，而一个网络数据包从客户端到服务器可能需要毫秒甚至秒级，网络I/O模型的演进，本质上是为了解决“如何高效地等待慢速的网络数据”

